

第 48 讲：CNN 基础

Kernel、Convolution、Feature Map 与参数共享到底是什么

《面向 LLM / Agent Systems 科研的深度学习零基础课》

本讲定位

前 47 讲已经把 Tensor、Linear、Activation、Backpropagation、Optimizer、Generalization、Normalization 逐步打通。本讲开始接触另一类经典深度学习架构：**Convolutional Neural Network (CNN，卷积神经网络)**。

CNN 不是本课程最终重点，因此不会深入 ResNet、检测、分割等完整视觉体系。本讲唯一目标是：**建立“局部连接 + 参数共享 + Feature Map”的底层直觉**，让你以后看到 Conv2d、Kernel、Channel、Stride、Padding、Pooling、Feature Map 时知道每个数字到底在做什么，并理解 CNN 为什么曾经长期统治视觉模型。

0. 学完以后，你应该能做到什么？

- 区分灰度图、彩色图、Batch 图像在 PyTorch 中的 Shape。
- 真正解释 Kernel / Filter 是什么，为什么一个小矩阵可以在整张图上重复滑动。
- 手算一个 5×5 输入经过 3×3 Kernel 后的 Feature Map。
- 解释为什么 PyTorch 的 Conv2d 严格数学上更接近 cross-correlation，但社区仍称 convolution。
- 用公式计算 Padding、Stride 对输出 Height / Width 的影响。
- 看懂 Conv2d 的 Weight Shape: $(C_{out}, C_{in}, K_h, K_w)$ 。
- 解释 Parameter Sharing 为什么让 CNN 参数量远小于把整张图 Flatten 后做巨大 Linear。
- 理解 Max Pooling、Receptive Field、Translation Equivariance 的基本直觉。
- 看懂一个最小 CNN 的完整 Shape Flow，并知道它与 MLP / Transformer 的关键差异。

【科研连接】

你以前在 LLM 课程里已经见过“参数共享”：同一套 Transformer Weight 会被应用到不同 Token 位置。本讲 CNN 会让你看到另一种更强的结构约束：**同一个小 Kernel 被重复应用到所有空间位置**。这种“结构先验 (inductive bias)”是理解 CNN、Transformer 以及不同模型为什么擅长不同数据结构的入口。

1 为什么图像不适合一上来就全部 Flatten?

假设一张 RGB 图像大小为：

$$32 \times 32 \times 3$$

一共有：

$$32 \times 32 \times 3 = 3072$$

个输入数字。

如果直接 Flatten 成长度 3072 的向量，再用 Linear 映射到 16384 个输出特征，则仅 Weight 就需要：

$$3072 \times 16384 = 50,331,648$$

个参数，再加 Bias 共约 5035 万。

【原理】

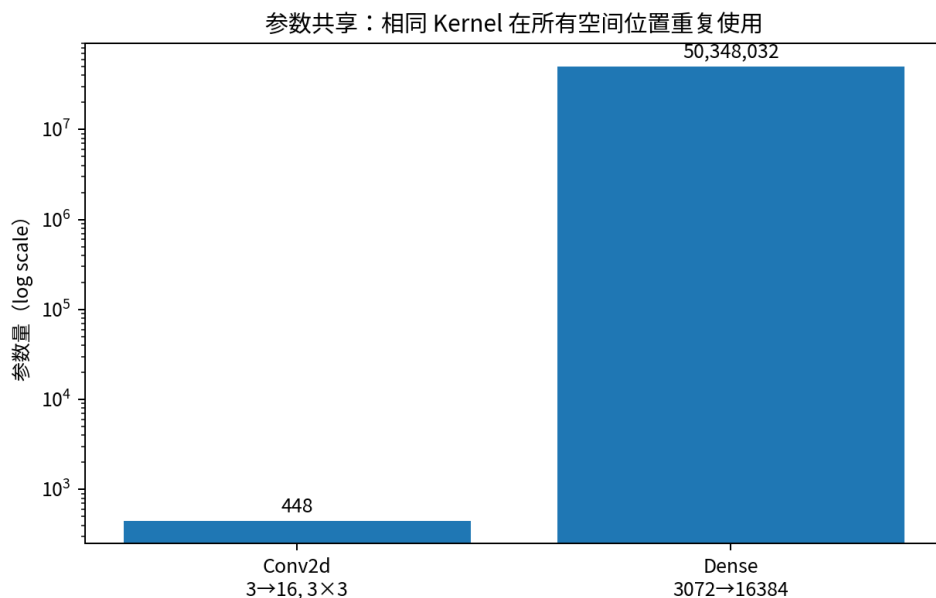
图像不是“3072 个互不相关的数字”。图像具有非常强的**空间局部结构**：相邻像素通常比相距很远的像素关系更直接；一条边缘、一块纹理、一个角点都可以在不同空间位置重复出现。

CNN 的核心策略不是“把所有像素一次性彼此连接”，而是：

1. 每次只看一个很小的局部区域；
2. 用同一组局部 Weight 扫过整张图；
3. 把每个位置的局部响应组成新的 Feature Map。

【通俗解释】

可以把 MLP 想成：每个位置都雇一套完全独立的检测器。CNN 则像只训练一套“ 3×3 小探测器”，然后让它走遍整张图。因为“检测竖边”这件事发生在左上角还是右下角，本质规则可以共享，所以没有必要为每个位置重新学习一套 Weight。



2 图像 Tensor：先把 Shape 看懂

一张灰度图可以先写成二维矩阵：

$$X \in \mathbb{R}^{H \times W}$$

例如：

$$28 \times 28$$

表示 28 行、28 列像素。

彩色 RGB 图像多一个 Channel 维。PyTorch 默认采用：

$$(C, H, W)$$

所以一张 32×32 RGB 图像通常是：

$$(3, 32, 32)$$

加入 Batch 后：

$$X.shape = (N, C, H, W)$$

其中：

- N ：Batch Size；
- C ：Channel 数；
- H ：Height；
- W ：Width。

【具体数字例子】

如果：

$$X.shape = (16, 3, 224, 224)$$

含义不是“16 维图像”，而是：一批 16 张图，每张 3 个 Channel，每个 Channel 的空间大小为 224×224 。

【常见误区 / 风险】

不同库的 Channel 顺序可能不同。PyTorch 常用 NCHW；很多图像文件或 NumPy / OpenCV 场景常见 HWC 或 NHWC。以后看到卷积 Shape 错误，第一件事就是确认 Channel 到底在哪个 Axis。

3 Kernel / Filter 到底是什么？

最简单的二维 Kernel 就是一个很小的 Weight Matrix。例如：

$$K = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

它的 Shape 是：

$$3 \times 3$$

这个 Kernel 会覆盖输入图像的一个 3×3 Patch，对应元素相乘，再全部求和，得到一个输出数字。

【原理】

一个卷积输出值，本质上仍然是你已经学过的：**局部 Dot Product**。

区别只是：这个 Dot Product 不对整张图做，而是对一个局部 Patch 做；然后同一个 Kernel 被移动到下一个位置，继续做同样的 Dot Product。

4 用一个 5×5 输入从头手算一次卷积

取输入：

$$X = \begin{bmatrix} 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 \end{bmatrix}$$

以及前面的 3×3 Kernel：

$$K = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

当 Kernel 覆盖左上角第一个 Patch：

$$P_1 = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

输出为：

$$\sum (P_1 \odot K) = 0$$

向右移动一格，Patch 变成：

$$P_2 = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 0 & 1 \\ 0 & 0 & 1 \end{bmatrix}$$

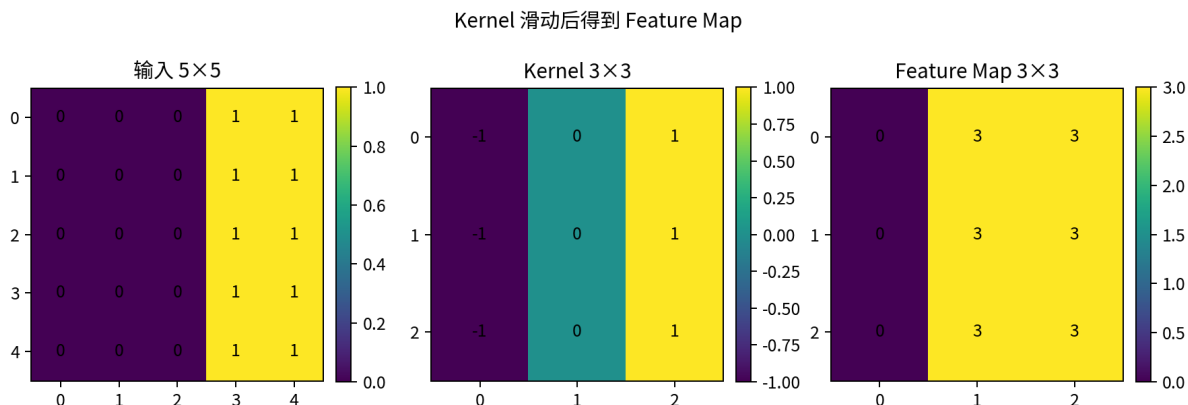
则：

$$\begin{aligned} P_2 \odot K &= \begin{bmatrix} 0 & 0 & 1 \\ 0 & 0 & 1 \\ 0 & 0 & 1 \end{bmatrix} \odot \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix} \\ &\Rightarrow 1 + 1 + 1 = 3 \end{aligned}$$

把 Kernel 滑遍所有合法位置，得到：

$$\begin{bmatrix} 0 & 3 & 3 \\ 0 & 3 & 3 \\ 0 & 3 & 3 \end{bmatrix}$$

这张新的二维矩阵就是一个 **Feature Map (特征图)**。



【通俗解释】

这里 Kernel 对“左边是 0、右边是 1”的局部结构响应很大，因此可以把它理解成一个简化的“垂直边缘检测器”。但真正 CNN 中的 Kernel 通常不是人手写的，而是通过 Backpropagation 从数据中自动学习。

5 Convolution 还是 Cross-Correlation? 一个必须知道的命名细节

严格的数学 convolution 会先把 Kernel 上下、左右翻转再滑动。但 PyTorch 的：

`nn.Conv2d / F.conv2d`

通常**不会翻转 Kernel**，因此数学上更接近 **cross-correlation (互相关)**。

实验使用一个非对称 Kernel 后：

- 手写“不翻转”的 cross-correlation 与 PyTorch 最大误差：0；
- 真正翻转 Kernel 的严格 convolution 与 PyTorch 最大差异：5。

【常见误区 / 风险】

以后论文、代码、框架里说“Convolution”，通常不需要纠结这个命名。深度学习中 Kernel 是可学习参数：如果翻转规则在整个系统中一致，模型可以学到相应 Weight。你需要知道的是**实现约定**，而不是把时间花在术语争论上。

6 Feature Map 是什么?

Kernel 在不同位置计算出的所有响应按照空间位置重新排成矩阵，就是 Feature Map。

一个 Feature Map 可以理解为：

“这个 Kernel 所代表的局部模式，在图像各个位置响应有多强。”

如果一个 CNN 层有很多不同 Kernel，就会产生很多输出 Feature Maps，也就是很多输出 Channels。

【科研连接】

这里已经出现一个与 Attention 很不同的机制：CNN 的 Kernel Weight 是学出来后在所有空间位置共享的固定局部变换；Attention 的权重则会根据当前 Query/Key 内容动态计算。两者都在“组合局部表示”，但归纳偏置完全不同。

7 Stride: Kernel 每次移动几格？

Stride (步幅) 决定 Kernel 每次移动多少个像素。

- Stride=1：每次移动 1 格；
- Stride=2：每次跳 2 格；
- Stride 越大，输出空间尺寸通常越小。

如果输入 Height 为 H ，Kernel Size 为 K ，Padding 为 P ，Stride 为 S ，且先假设 Dilation=1，则：

$$H_{out} = \left\lfloor \frac{H + 2P - K}{S} \right\rfloor + 1$$

Width 同理：

$$W_{out} = \left\lfloor \frac{W + 2P - K}{S} \right\rfloor + 1$$

【具体数字例子】

输入 7×7 ，Kernel=3，Stride=2，Padding=1：

$$H_{out} = \left\lfloor \frac{7 + 2 - 3}{2} \right\rfloor + 1 = \lfloor 3 \rfloor + 1 = 4$$

所以输出空间大小为：

$$4 \times 4$$

实验与 PyTorch 完全一致。

8 Padding: 为什么在图像边缘补 0？

如果 7×7 输入经过 3×3 Kernel、Stride=1、Padding=0：

$$7 \rightarrow 5$$

空间尺寸缩小。

若 Padding=1：

$$7 \rightarrow 7$$

就可以保持 Height / Width 不变。

【通俗解释】

Padding 可以理解成：在原图外围临时补一圈数，让 Kernel 有机会把“图像边缘像素”也放到 Kernel 中央附近处理，同时控制输出 Shape。

【常见误区 / 风险】

“same padding”不是一句永远等于 $P = (K - 1)/2$ 的魔法规则。对奇数 Kernel、Stride=1 时这个直觉很好用；Stride、偶数 Kernel、Dilation 变化后必须回到输出 Shape 公式或框架定义。

9 多 Channel 卷积：RGB 图像怎么卷？

一张 RGB 图像有：

$$C_{in} = 3$$

个输入 Channel。

如果我们想得到一个输出 Feature Map，那么这个输出 Kernel 不能只是二维 3×3 ，而必须同时覆盖所有输入 Channels：

$$(C_{in}, K_h, K_w)$$

如果要产生 C_{out} 个输出 Channels，PyTorch Conv2d Weight Shape 是：

$$W.shape = (C_{out}, C_{in}, K_h, K_w)$$

Bias：

$$b.shape = (C_{out},)$$

完整输入输出：

$$(N, C_{in}, H, W) \rightarrow (N, C_{out}, H_{out}, W_{out})$$

【原理】

一个输出 Channel 的某个空间位置，会对**所有输入 Channels** 的**局部 Patch** 分别做乘加，然后把结果全部求和，再加这个输出 Channel 自己的 Bias。

实验中：

$$X.shape = (2, 4, 4)$$

$$W.shape = (2, 2, 3, 3)$$

得到：

$$Y.shape = (2, 2, 2)$$

手写多 Channel 卷积与 PyTorch 最大误差为 0。

【具体数字例子】

`nn.Conv2d(3, 16, kernel_size=3)` 的 Weight Shape 是：

$$(16, 3, 3, 3)$$

含义不是“16 个 3×3 Kernel”，而更准确地说：**16 个输出 Filter**，每个 Filter 都包含 3 个输入 Channel 的 3×3 Weight。

10 Parameter Sharing: CNN 为什么参数量可以这么少?

Conv2d 参数量公式:

$$C_{out} \times C_{in} \times K_h \times K_w + C_{out}$$

后面的 C_{out} 来自 Bias。

实验:

$$\text{Conv2d}(3 \rightarrow 16, K = 3)$$

参数量:

$$16 \times 3 \times 3 \times 3 + 16 = 448$$

注意: 无论输入空间是 32×32 还是 224×224 , **这个卷积层本身的参数量仍然是 448**。变化的是计算次数和输出 Feature Map 大小, 不是 Kernel 参数数量。

这就是 Parameter Sharing:

同一套 448 个参数被重复应用到很多不同空间位置。

实验中, 若把 $32 \times 32 \times 3$ 直接 Dense 映射到 $32 \times 32 \times 16$, 需要:

$$50,348,032$$

个参数, 是卷积的:

$$112,384 \text{ 倍}$$

【科研连接】

Parameter Sharing 不是“免费午餐”, 它其实是在向模型加入一个强假设: **同一种局部模式不管出现在图像哪个位置, 都值得用同一套检测规则**。这叫 Inductive Bias。CNN 的高效率, 很大部分来自这个假设恰好非常适合图像。

11 为什么 Kernel Weight 可以自动学成“边缘检测器”?

前面我们手工写了边缘 Kernel, 但真实训练中通常只随机初始化 Kernel。

训练过程仍然是你已经学过的:

$$\text{Forward} \rightarrow \text{Loss} \rightarrow \text{Backward} \rightarrow \text{Gradient} \rightarrow \text{Optimizer}$$

因为同一个 Kernel 在很多空间位置被重复使用, 所以它的某一个 Weight 会影响很多输出位置。Backward 时, 这些空间位置对同一个共享 Weight 的 Gradient 贡献会累加。

【通俗解释】

这和第 23 讲“一个变量被多条计算路径使用时, 梯度需要相加”完全是同一个原理。CNN 没有发明新的 Backpropagation; 它只是让同一个 Parameter 在空间上被复用了很多次。

12 Activation、堆叠卷积与 Receptive Field

一个常见 CNN Block 可以写成：

$$\text{Conv} \rightarrow \text{ReLU} \rightarrow \text{Conv} \rightarrow \text{ReLU}$$

如果没有 Activation，多层 Linear Convolution 仍然会受到“线性层叠可合并”的限制。ReLU 等 Activation 继续承担引入非线性的职责。

Receptive Field (感受野) 回答的是：

某个深层 Feature 的一个位置，理论上能受到原始输入多大范围的影响？

在最简单 stride=1、无 dilation 的情况下：

- 1 个 3×3 Conv：感受野 3×3 ；
- 再堆 1 个 3×3 Conv：感受野扩大到 5×5 ；
- 第 3 个：可扩大到 7×7 。

所以 CNN 虽然每一层只做 Local Operation，但通过堆叠很多层，深层单元最终可以整合越来越大的空间范围。

【常见误区 / 风险】

“卷积只看局部”描述的是单层 Kernel。深层 CNN 的有效感受野可以很大。不要把“局部连接”误解成整个网络永远只能理解一个小 Patch。

13 Pooling：为什么要把 Feature Map 变小？

Max Pooling 取局部窗口中的最大值。例如：

$$\begin{bmatrix} 1 & 3 \\ 4 & 6 \end{bmatrix} \rightarrow 6$$

实验中：

$$(1, 1, 4, 4) \xrightarrow{\text{MaxPool}(2,2)} (1, 1, 2, 2)$$

得到：

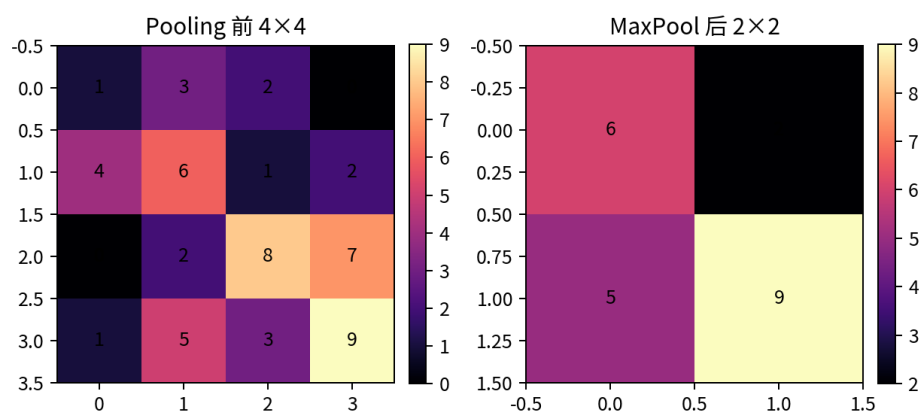
$$\begin{bmatrix} 6 & 2 \\ 5 & 9 \end{bmatrix}$$

Pooling 的作用直觉包括：

- 降低空间分辨率；
- 减少后续计算量；
- 让表示对很小的位置变化具有一定容忍度。

【常见误区 / 风险】

Pooling 会丢失空间信息，所以不是越多越好。现代架构中也经常用 Strided Convolution 或其他下采样方式代替传统 Pooling。



14 Translation Equivariance: CNN 不是自动“平移不变”

卷积有一个非常重要的直觉：如果局部模式整体移动，Feature Map 中相应的响应也会移动。

实验把一个点从：

$(4, 4)$

向右移动到：

$(4, 5)$

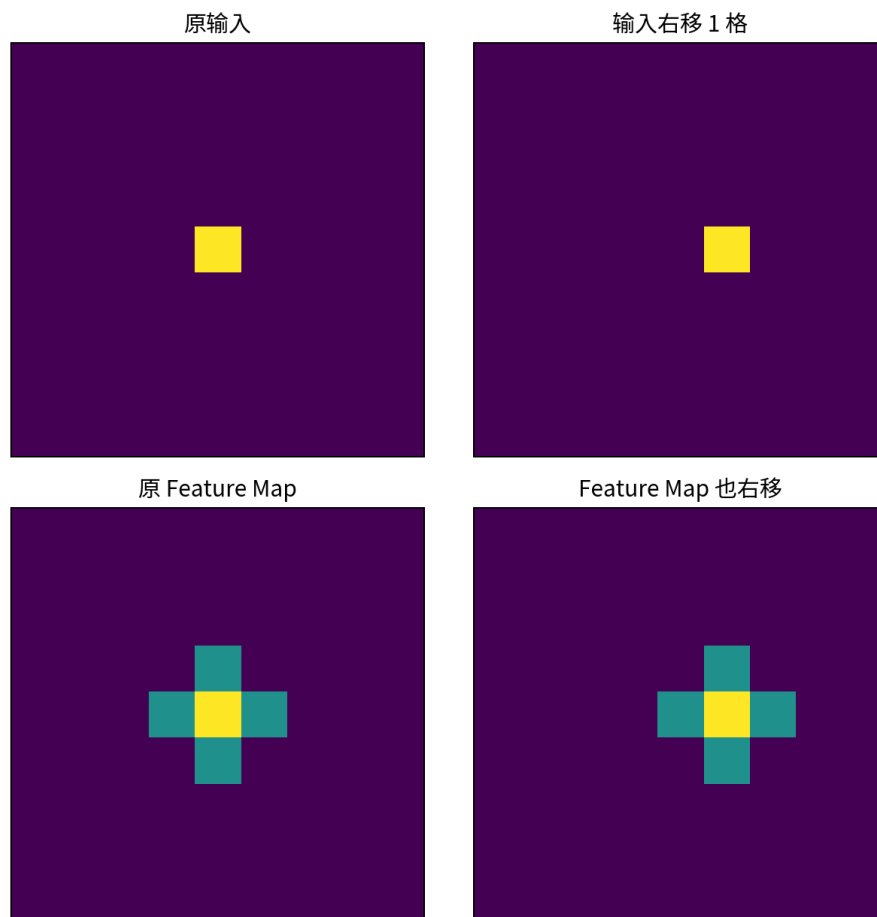
经过同一个 Kernel 后，最大响应位置也从：

$(4, 4) \rightarrow (4, 5)$

这更准确地叫：

Translation Equivariance

即输入平移，输出也以相应方式平移。

**【常见误区 / 风险】**

不要把 Equivariance 和 Invariance 混为一谈。

Equivariance: 输入移动，表示也跟着移动。

Invariance: 输入移动，但最终任务输出基本不变。

卷积天然更接近前者；Pooling、Global Pooling、数据增强和后续架构可能帮助获得一定平移不变性，但不是“只要用了 CNN 就完全不怕位置变化”。

15 第一次看完整 TinyCNN: Shape 一步一步走

我们构造一个非常小的模型：

$$Input \rightarrow Conv \rightarrow ReLU \rightarrow MaxPool \rightarrow Flatten \rightarrow Linear \rightarrow Logits$$

输入 Batch:

$$X.shape = (5, 1, 8, 8)$$

第一层:

$$Conv2d(1, 4, K = 3, P = 1)$$

得到:

$$(5, 4, 8, 8)$$

ReLU 不改 Shape:

$$(5, 4, 8, 8)$$

MaxPool 2×2 :

$$(5, 4, 4, 4)$$

Flatten:

$$(5, 64)$$

最后:

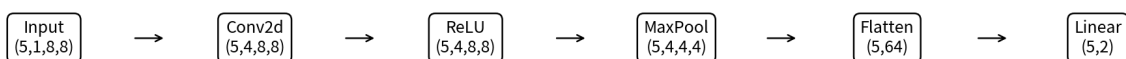
$$Linear(64, 2)$$

得到:

$$(5, 2)$$

也就是每个样本产生 2 个分类 Logits。

TinyCNN : 每一步都先追踪 Shape



参数量:

$$Conv : 4 \times 1 \times 3 \times 3 + 4 = 40$$

$$FC : 64 \times 2 + 2 = 130$$

总计:

$$170$$

【代码实验】

```
class TinyCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv = nn.Conv2d(1, 4, kernel_size=3, padding=1)
```

```

self.relu = nn.ReLU()
self.pool = nn.MaxPool2d(2)
self.fc = nn.Linear(4 * 4 * 4, 2)

def forward(self, x):
    x = self.conv(x)      # (N,1,8,8) -> (N,4,8,8)
    x = self.relu(x)      # Shape 不变
    x = self.pool(x)      # -> (N,4,4,4)
    x = torch.flatten(x, 1) # -> (N,64)
    return self.fc(x)     # -> (N,2)

```

16 CNN、MLP、Transformer：到底差在哪里？

维度	MLP / Linear	CNN	Transformer / Attention
连接方式	通常全连接特征	局部空间连接	Token / Patch 之间内容相关的动态连接
参数共享	Linear 对 Batch/位置复用	Kernel 在空间位置共享	Q/K/V/FFN Weight 对 Token 位置共享
交互范围	取决于输入表示	单层局部，堆叠扩大感受野	Self-Attention 可直接建立长距离交互
Inductive Bias	较弱	强局部性、平移结构先验	更灵活，强调内容关系
典型数据	表格/通用向量	图像/网格信号	序列、文本、Patch、通用 Token

【原理】

CNN 的核心并不是“会处理图片”这句话，而是：**它把局部性和参数共享写进了模型结构。**Transformer 的 Attention 则把“谁应该和谁交互”更多交给数据和内容本身动态决定。

【通俗解释】

CNN 像一支拿着同一副小放大镜、在整张图上扫描的巡检队；Attention 更像每个 Token 都能根据当前任务动态决定“我现在最应该看谁”。

17 与 LLM / Agent Systems 的关系

CNN 不是文本 LLM 的核心架构，但本讲仍直接服务你的长期方向：

- 多模态 LLM / Agent 的视觉输入常需要 Vision Encoder；CNN 仍是理解视觉 Backbone 的基本语言。
- Agent 处理截图、机器人视觉、视频帧、环境观察时，可能直接或间接使用卷积模块。
- Parameter Sharing、Tensor Shape、Backpropagation、Activation、Normalization、Regularization 等前面知识在 CNN 中全部再次出现，说明不同架构底层仍共享同一套深度学习机制。

- 对比 CNN 与 Transformer，可以更清楚理解“架构的 Inductive Bias”这个科研概念：不是只有参数数量决定模型能力，**连接结构本身就是假设**。

【科研连接】

未来做 Agent Systems 论文，如果视觉感知不是研究问题本身，不应为了“更深度学习”而把大量时间花在设计 CNN Backbone。你需要达到的是：能读懂视觉输入模块的 Shape、主要结构和实验变量，知道它会不会成为 confounder，并能在需要时做最小复现实验。

18 科研读图与 Ablation：看到 CNN 论文应该先问什么？

看到一个 CNN 模块，不要只记模型名字。优先问：

1. 输入 Shape 是什么？Channel 在哪里？
2. Kernel Size、Stride、Padding 分别是多少？
3. 每层 C_{in} 和 C_{out} 怎样变化？
4. 空间分辨率在哪些位置下降？
5. 参数量增长来自 Channel 还是空间尺寸？
6. 是否存在 Pooling、Residual、Normalization？
7. 训练时 Kernel 是冻结还是更新？
8. Ablation 改变了 Kernel Size、Depth、Width 还是 Downsampling？
9. 比较不同架构时是否控制了参数量、训练预算和数据增强？

【常见误区 / 风险】

CNN 论文中一个很常见的混淆是：A 模型效果更好，同时它又更深、更宽、训练更久、数据增强更多。此时不能仅凭最终 Accuracy 断言“某个卷积设计更优”。科研上仍需要控制变量和 Ablation。

19 常见报错怎么从 Shape 定位？

- **Expected input to have C channels...**：检查 NCHW / NHWC 是否搞反，或上一层 C_{out} 是否匹配下一层 C_{in} 。
- **Kernel size can't be greater than input size**：空间尺寸已经太小，Kernel 太大或前面下采样太多。
- **Linear matmul shape mismatch**：通常是 Conv/Pool 后 Flatten Size 算错；先打印 `x.shape`，不要盲猜 `in_features`。
- Output H/W 不符合预期：重新代入 H_{out}, W_{out} 公式检查 Stride / Padding。

【代码实验】

```
# 零基础排错时，先打印 Shape:
print("input:", x.shape)
x = self.conv(x)
print("after conv:", x.shape)
x = self.pool(x)
```

```
print("after pool:", x.shape)
x = torch.flatten(x, 1)
print("after flatten:", x.shape)
```

20 本讲最小闭环：把 CNN 还原成你已经学过的东西

到这里，CNN 不应该再是一个新的黑盒。它其实仍然可以还原成前面已经掌握的机制：

$$Local\ Patch \xrightarrow{\text{局部乘加}} Feature\ Map$$

$$Shared\ Kernel \xrightarrow{\text{空间重复使用}} Parameter\ Sharing$$

$$Conv \rightarrow Activation \rightarrow Pool/Downsample \rightarrow \text{更高层 Feature}$$

训练仍然是：

$$Forward \rightarrow Loss \rightarrow Backward \rightarrow Optimizer$$

5 分钟口述自检

不看讲义，尝试回答：

1. 为什么 3×3 Kernel 最终只产生一个输出数字？
2. Feature Map 中每个位置的数字表示什么？
3. 为什么 Conv2d 的参数数量不随输入 Height / Width 增加？
4. $(N, 3, 32, 32)$ 经 $Conv2d(3, 16, 3, padding=1)$ 后 Shape 是多少？为什么？
5. PyTorch Conv2d Weight 为什么是 $(C_{out}, C_{in}, K_h, K_w)$ ？
6. Stride=2 为什么通常会减小空间尺寸？
7. MaxPool 有可学习 Weight 吗？
8. Translation Equivariance 与 Invariance 有什么区别？
9. CNN 与 Transformer 最大的结构性差异是什么？

如果其中 7 个以上能脱离公式自己解释，本讲就基本达到目标。

【科研连接】

本课程到 CNN 的目标是“理解架构语言”，不是变成视觉方向研究员。下一步我们会转向 RNN / LSTM：从空间局部性转向序列状态与时间依赖，最终再回到为什么 Transformer 改变了序列建模方式。